

Inspection HTTPS Android

Désactivation du SSL Pinning avec Objection + Proxy

Cours : Sécurité des applications mobiles

Analyste : Omar Haouani

Outils : Objection 1.12.4, Frida 17.9.5

Proxy : Burp Suite / mitmproxy

Date : 20 mai 2026

Table des matières

1	Introduction	3
1.1	Objectifs	3
1.2	Prérequis	3
2	Environnement Technique	3
3	Phase 1 : Installation et Vérification d'Objection	4
3.1	Objection -help	4
4	Phase 2 : Vérification de l'Environnement Frida	5
4.1	Vérification des versions	5
4.2	Vérification de l'architecture CPU	5
4.3	Vérification des processus	6
5	Phase 3 : Configuration du Proxy et Installation du Certificat CA	7
5.1	Configuration de Burp Suite / mitmproxy	7
5.2	Installation du certificat CA	7
6	Phase 4 : Désactivation du SSL Pinning avec Objection	7
6.1	Lancement d'Objection sur l'application cible	7
6.2	Commande de bypass SSL Pinning	7
6.3	Exemple complet	7
7	Phase 5 : Validation de l'Interception	8
7.1	Vérifications à effectuer	8
7.2	Résultats attendus	8
8	Dépannage (FAQ)	8
9	Schéma d'Exploitation	9
10	Checklist Rapide	9
11	Conclusion	9

1 Introduction

Ce laboratoire porte sur l'inspection du trafic HTTPS d'applications Android en contournant le mécanisme de SSL Pinning à l'aide d'**Objection**. Objection est un outil d'instrumentation mobile basé sur Frida qui simplifie considérablement le processus de bypass.

1.1 Objectifs

- Installer et configurer Objection et Frida
- Préparer l'appareil Android (émulateur rooté)
- Configurer un proxy intercepteur (Burp Suite / mitmproxy)
- Désactiver le SSL Pinning avec Objection
- Valider l'interception du trafic HTTPS
- Résoudre les problèmes courants

1.2 Prérequis

- Python 3.8+ et pip
- ADB (Android Platform Tools)
- Appareil Android rooté ou émulateur
- Frida et frida-server (versions alignées)
- Proxy intercepteur (Burp Suite / mitmproxy)

2 Environnement Technique

Composant	Spécification
Objection	1.12.4
Frida Client	17.9.5
Frida Server	17.9.5
Appareil	Émulateur Android (emulator-5554)
Architecture	x86_64
Proxy	Burp Suite / mitmproxy

3.1 Objection –help

```
C:\Windows\System32>objection --help
Usage: objection [OPTIONS] COMMAND [ARGS]...

  .  .  -  -  -  .  .  .
  |  |  |  |  |  |  |  |  |
  |__|(object)inject(ion)

Runtime Mobile Exploration
by: @leonjza from @sensepost

Options:
-N, --network          Connect using a network connection instead of USB.
-h, --host TEXT        [default: 127.0.0.1]
-P, --port INTEGER     [default: 27042]
-ah, --api-host TEXT   [default: 127.0.0.1]
-ap, --api-port INTEGER [default: 8888]
-n, --name TEXT        Name or bundle identifier to attach to.
-S, --serial TEXT      A device serial to connect to.
-d, --debug            Enable debug mode with verbose output.
-s, --spawn            Spawn the target.
-p, --no-pause         Resume the target immediately.
-f, --foremost         Use the current foremost application.
--debugger            Enable the Chrome debug port.
--uid TEXT             Specify the uid to run as (Android only).
--help                Show this message and exit.

Commands:
api          Start the objection API server in headless mode.
patchapk     Patch an APK with the frida-gadget.so.
patchipa     Patch an IPA with the FridaGadget dylib.
run          Run a single objection command.
signapk      Zipalign and sign an APK with the objection key.
start        Start a new session
version      Prints the current version and exits.

C:\Windows\System32>objection version
objection: 1.12.4
```

FIGURE 1 – Aide d'Objection et version

```
objection --help
objection version
```

Résultat : Objection 1.12.4 est installé.

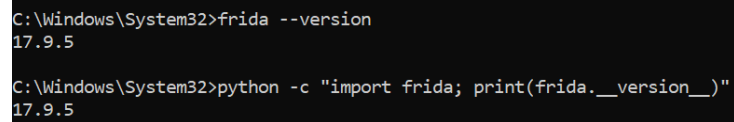
Commandes principales disponibles :

- **start** - Démarrer une session
- **run** - Exécuter une commande unique
- **patchapk** - Patcher un APK avec frida-gadget

- `signapk` - Signer un APK
- `api` - Démarrer le serveur API

4 Phase 2 : Vérification de l'Environnement Frida

4.1 Vérification des versions



```
C:\Windows\System32>frida --version
17.9.5

C:\Windows\System32>python -c "import frida; print(frida.__version__)"
17.9.5
```

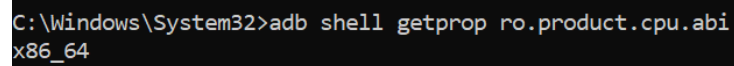
FIGURE 2 – Vérification des versions Frida

```
frida --version
python -c "import frida; print(frida.__version__)"
```

Résultats :

- Frida CLI : 17.9.5
- Frida Python : 17.9.5

4.2 Vérification de l'architecture CPU



```
C:\Windows\System32>adb shell getprop ro.product.cpu.abi
x86_64
```

FIGURE 3 – Architecture CPU de l'émulateur

```
adb shell getprop ro.product.cpu.abi
```

Résultat : `x86_64`

4.3 Vérification des processus

```
C:\Windows\System32>frida-ps -U
  PID  Name
-----
14318  Calendar
14153  Chrome
14173  Clock
12507  ConverterTabsJava
11290  FragmentsLab
  1966  Google
13718  JNIDemo
  4116  Messages
14458  Phone
  1388  SIM Toolkit
14073  Settings
  4571  abb
   581  adbd
   505  android.hardware.audio.service
   524  android.hardware.authsecret-service.example
1158  android.hardware.biometrics.fingerprint-service.ranchu
   520  android.hardware.bluetooth-service.default
   509  android.hardware.camera.provider.ranchu
   510  android.hardware.camera.provider@2.7-service-google
   532  android.hardware.cas-service.example
   539  android.hardware.contexthub-service.example
   565  android.hardware.drm-service.widevine
   275  android.hardware.gatekeeper-service.nonsecure
1270  android.hardware.gnss-service.ranchu
   511  android.hardware.graphics allocator-service.ranchu
   521  android.hardware.graphics.composer3-service.ranchu
   514  android.hardware.health-service.example
   522  android.hardware.identity-service.example
   523  android.hardware.lights-service.example
   515  android.hardware.media.c2@1.0-service-goldfish
   548  android.hardware.neuralnetworks-service-sample-all
   549  android.hardware.neuralnetworks-service-sample-limited
   550  android.hardware.neuralnetworks-shim-service-sample
   551  android.hardware.power-service.example
   555  android.hardware.power.stats-service.example
   516  android.hardware.radio-service.ranchu
```

FIGURE 4 – Liste des processus Android (frida-ps -U)

```
frida-ps -U
```

Observation : L'émulateur (emulator-5554) est correctement connecté.

5 Phase 3 : Configuration du Proxy et Installation du Certificat CA

5.1 Configuration de Burp Suite / mitmproxy

1. Lancer Burp Suite → Proxy → Proxy Listeners → Add (0.0.0.0 :8080)
2. Sur l'émulateur : Paramètres → Wi-Fi → Modifier le réseau
3. Proxy manuel : IP du PC hôte, port 8080

5.2 Installation du certificat CA

1. Burp Suite : Proxy → Options → Import/export CA certificate → Export
2. Transférer le certificat : `adb push cacert.der /sdcard/`
3. Paramètres → Sécurité → Installer depuis le stockage
4. Nommer le certificat et valider

6 Phase 4 : Désactivation du SSL Pinning avec Objection

6.1 Lancement d'Objection sur l'application cible

```
objection -g com.target.app start
```

6.2 Commande de bypass SSL Pinning

```
android sslpinning disable
```

Explication : Cette commande active automatiquement plusieurs hooks pour contourner le SSL Pinning :

- Hook de SSLContext.init
- Hook de X509TrustManager
- Hook de TrustManagerImpl (conscript)
- Hook de CertificatePinner (OkHTTP)

6.3 Exemple complet

```
objection -g com.pwnsec.firestorm start
android sslpinning disable
```

7 Phase 5 : Validation de l'Interception

7.1 Vérifications à effectuer

1. Dans le proxy (Burp/mitmproxy), observer le trafic HTTP/HTTPS
2. Vérifier que les requêtes sont déchiffrées (affiche en clair)
3. Tester l'application normalement (connexion, formulaires)

7.2 Résultats attendus

Vérification	Statut
Proxy intercepte le trafic HTTPS déchiffré Application fonctionne Pas d'erreur SSL	

8 Dépannage (FAQ)

Problème	Solution
Objection ne détecte pas l'appareil	Vérifier <code>adb devices</code> et redémarrer frida-server
android sslpinning disable ne fonctionne pas	Vérifier que l'application utilise bien les classes Java standards
Le proxy n'intercepte rien	Vérifier la configuration du proxy et le certificat CA
Erreur de version frida-server	Aligner les versions client et serveur
Pinning natif résiste	Utiliser un script Frida personnalisé pour hooker les fonctions natives

9 Schéma d'Exploitation

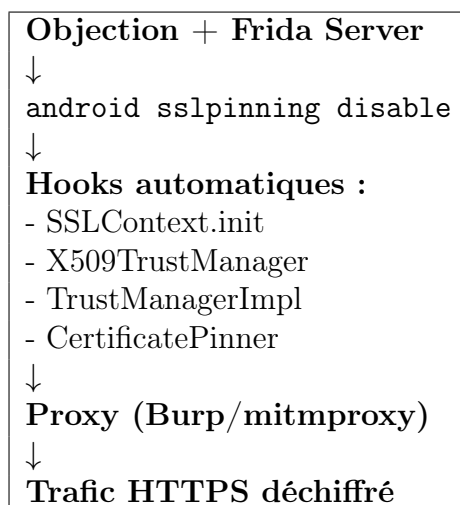


FIGURE 5 – Flux d'exploitation avec Objection

10 Checklist Rapide

- x Objection, Frida, frida-server installés et alignés
- x Proxy opérationnel (Burp/mitmproxy)
- x Certificat CA installé sur l'appareil
- x Lancement Objection (spawn ou attach)
- x Commande `android sslpinning disable` exécutée
- x Trafic HTTPS de l'app visible dans le proxy

11 Conclusion

Ce laboratoire a démontré :

1. La simplicité d'Objection pour le bypass du SSL Pinning
2. L'efficacité de la commande `android sslpinning disable`
3. L'importance de combiner proxy et instrumentation
4. La nécessité de comprendre les mécanismes sous-jacents

Objection est un outil puissant qui rend accessible l'inspection du trafic HTTPS à tous les niveaux de compétence.

12 Ressources

- [Objection sur GitHub](#)
- [Documentation Frida](#)
- [Burp Suite](#)
- [mitmproxy](#)

Document réalisé dans le cadre du cours de Sécurité des applications mobiles
Omar Haouani - 20 mai 2026